# Text Representation:
## Word Embedding

**MSIS 521**

**Professor Hossein Ghasemkhani**

FOSTER SCHOOL OF BUSINESS
UNIVERSITY OF WASHINGTON

1

## Agenda

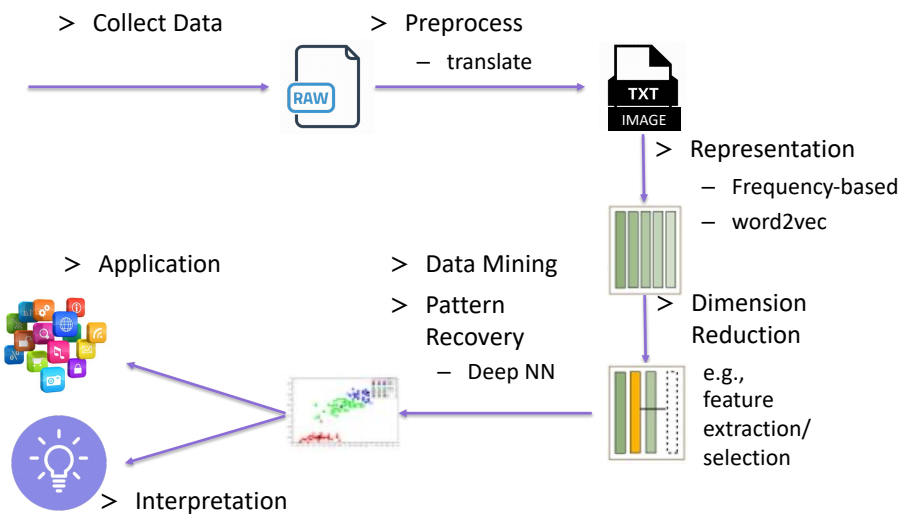
- Unstructured Data Analytics Process
- Word embedding – frequency based
  - Word level information
    - Tokenization
    - Bag of words
    - Stemming
    - Lemmatization
    - TF-IDF
  - Phrase level information
    - N-grams
  - Part-of-speech level information
    - POS
    - Parse Tree

FOSTER SCHOOL OF BUSINESS

2

2

# Unstructured Data Analytics Process




FOSTER SCHOOL OF BUSINESS

3

3

# Text Representation

FOSTER SCHOOL OF BUSINESS

4

4

## What Is Text?

- Text document is a sequence of
  - Characters
  - **Words**
  - Phrases
  - Parts-of-speech
- It seems natural to think of a text document as a sequence of words
  - A fine-grained unit
  - Naturally separated by spaces or punctuation
  - Meaningful (characters are not)

FOSTER SCHOOL OF BUSINESS

5

5

## Word Embedding

Vector representation of a word:

- Frequency-based
- Prediction-based
  - word2vec

FOSTER SCHOOL OF BUSINESS

6

6

## Text Representation: Word Level

7

7

## Tokenization - A Simple Representation

- Language Text
  - d1 = “David loves dogs”
  - d2 = “Dogs are ok with David”
  - d3 = “Cats love rain”
- Tokenization: split text into pieces, called tokens
  - Note: token can be more general, it can be a character, a word, a sentence, etc.
- When a token is a word, it results in a set of words
  - d1 = (“david”, “loves”, “dog”)
  - d2 = (“dogs”, “are”, ”ok”, “with”, “david”)
  - d3 = (“cats”, “love”, “rain”)

8

8

## Practice 1: Tokenization

- There are many tokenizers based on different rules
- https://www.nltk.org/py-modindex.html

```
import nltk
nltk.download('punkt') # tokenizing
```

```
d1 = "He is a good guy, he is not bad"
d2 = "feet wolves cooked boys girls ,!<@!"
d3 = "He is not a good guy, he is bad"
```

```
c1 = [d1, d2, d3] # list of docs

# tokenize
token_d1 = nltk.word_tokenize(d1)
print(token_d1)
```

```
tokenizer2 = nltk.tokenize.WhitespaceTokenizer()
token_d12 = tokenizer2.tokenize(d1)
print(token_d12)
```

- Problem?

FOSTER SCHOOL OF BUSINESS

9

9

## Problem: Not Mathematical!

- It is still text!! Can't use math to compute, manipulate and analyze
  - s1 = ("david", "loves", "dog")
  - s2 = ("dogs", "are", "ok", "with", "david")
  - s3 = ("cats", "love", "rain")
- We need to transform text into a numerical representation, so we can use math and computer programs

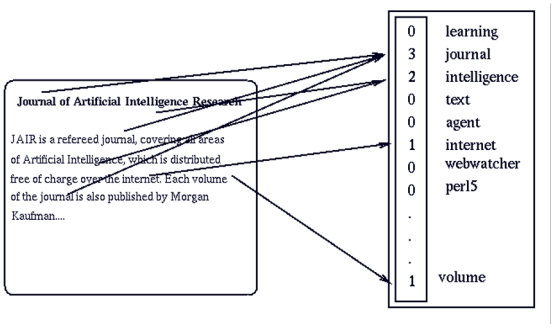
FOSTER SCHOOL OF BUSINESS

10

10

## Another Simple Representation: Bag of Words (BOW)

- BoW is a simplified representation of text that focuses on the occurrences of words in a document, disregarding grammar and word order.
- Example:




- No order
- The vector is defined to include the **union** of the words in the collection of the documents.

11

11

## Document-Term Matrix (DTM)

- A fundamental representation in traditional text mining and information retrieval
  - Term: typically, a single word (unigram), or an **n-gram** phrase like "data mining"
  - Document: a unit collection of text, such as an article, book, or tweet
  - Can be large, terms are often 50k or larger, documents can be in the billions (www).
  - Values in a DTM can be **binary (presence/absence)** or **count-based (term frequency).**
  - Example: 5 docs, 6 terms → a 5x6 matrix
- Also, referred to as TDM.

$$D_1 = (d_{i1}, d_{i2}, \ldots, d_{it})$$

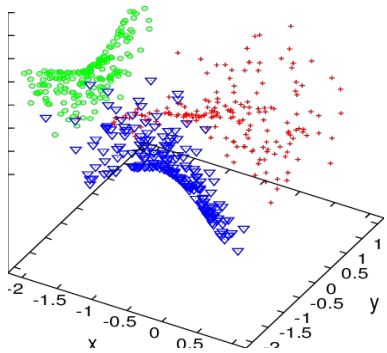
| | Database | SQL | Index | Regression | Likelihood | linear |
|---|---|---|---|---|---|---|
| D1 | 24 | 21 | 9 | 0 | 0 | 3 |
| D2 | 32 | 10 | 5 | 0 | 3 | 0 |
| D3 | 12 | 16 | 5 | 0 | 0 | 0 |
| D4 | 6 | 7 | 2 | 0 | 0 | 0 |
| D5 | 43 | 31 | 20 | 0 | 3 | 0 |

12

12

## Math Visualization

- Each term is represented as a separate **variable** having numeric weight (importance).
- When |terms|=3, each document is a dot in a 3-D Euclidean space where each axis represents a term.




FOSTER SCHOOL OF BUSINESS

13

13

## Practice 2: BoW

- Use a term-document matrix to implement a simple bag-of-words representation

```
# BOW
from sklearn.feature_extraction.text import CountVectorizer
vectorizer1 = CountVectorizer()
vectorizer1.fit(c1) # learn vocabularies from docs

print(vectorizer1.vocabulary_)
```

```
v1 = vectorizer1.transform(c1)
print(v1.toarray())
```

FOSTER SCHOOL OF BUSINESS

14

14

## Problem: Too Many Dimensions

- **Different forms** of a word but the **same meaning**
- Different forms of the same word are usually problematic for text data analysis, because they have different spelling, but similar meaning
  - E.g., learn, learns, learned, learning
  - E.g., dogs, dog
- Solution: **normalization**
  - Use the same token for different forms of the same word
  - Approach 1: stemming
  - Approach 2: lemmatization
  - …

15

15

## Stemming

- The process of transforming a word into its stem, base or root form (normalized form) by removing suffixes
- Example: Porter's stemmer nltk.stem.PorterStemmer
  - http://www.tartarus.org/~martin/PorterStemmer/

Step 1a

| | | | |
|---|---|---|---|
| sses → ss | caresses | → caress | |
| ies → i | ponies | → poni | |
| ss → ss | caress | → caress | |
| s → ø | cats | → cat | |

Step 1b

| | | | |
|---|---|---|---|
| (*v*)ing → ø | walking | → walk | |
| | sing | → sing | |
| (*v*)ed → ø | plastered | → plaster | |

Step 2 (for long stems)

| | | | |
|---|---|---|---|
| ational→ ate | relational | → relate | |
| izer→ ize | digitizer | → digitize | |
| ator→ ate | operator | → operate | |

Step 3 (for longer stems)

| | | | |
|---|---|---|---|
| al → ø | revival | → reviv | |
| able → ø | adjustable | → adjust | |
| ate → ø | activate | → activ | |
| … | | | |

16

16

## Stemming and Dimension Reduction

- Dimension Reduction

| | Tokens = N | Types = \|V\| |
|---|---|---|
| Switchboard phone conversations | 2.4 million | 20 thousand |
| Shakespeare | 884,000 | 31 thousand |
| Google N-grams | 1 trillion | 13 million |

- Limitation
  - Doesn't work on Irregular forms
    - feet ~ feet
  - Produces non-words
    - dummies ~ dummi

FOSTER SCHOOL OF BUSINESS

17

17

## Lemmatization

- Use a pre-defined database (dictionary) to lookup lemmas
- Example: WordNet lemmatizer nltk.stem.WordNetLemmatizer
  - dummies ~ dummy
  - feet ~ foot
- Limitation
  - Does not work for all forms without specifying parts of speech (cooked ~ cooked), and it is slower
- Which is better:
  - try both and find out which better
  - If meaning matters, lemmatization is preferred; if speed matters, stemming better

FOSTER SCHOOL OF BUSINESS

18

18

## Other Normalization:

- Capital letter: lowercase
  - Special case: Us, us, US
- Punctuation (.,!@#$%^&*~|}{_+:”?><`;’[]\): filter out
  - Special case?
- Acronyms
  - US, U.S, USA, usa, U.S.A
- Case by case, write case-specific rules for treatment.

19

19

## Practice 3: Normalization

- There are many stemmers and lemmatizers based on different rules
- https://www.nltk.org/py-modindex.html

```
# stemming
token_d2 = nltk.word_tokenize(d2.lower())

stemmer = nltk.stem.PorterStemmer()
stemmed_token_d2 = [stemmer.stem(token) for token in token_d2 if token.isalpha()]
print(stemmed_token_d2)
print(token_d2)
```

```
# lemmatizing
lemmatizer = nltk.stem.WordNetLemmatizer()
lemmatized_token_d2 = [lemmatizer.lemmatize(token) for token in token_d2 if token.isalpha()]
print(lemmatized_token_d2)
```

```
#
stemmed_token_d2 = []
for token in token_d2:
    if token.isalpha():
        stemmed.token_d2.append(stemmer.stem(token))
```

20

20

## Problem: Still Too Many Dimensions

- Drop less important words
- Step 1: Remove high-frequency words
  - Articles, prepositions, etc. (Examples: A, ABOUT, ABOVE, ACROSS, AFTER, AGAIN, AGAINST, ALL, ALMOST, ALONE, ALONG, ALREADY, IS, ARE…)
  - They are called **stop-words**:
    - do **not** carry information: do not help to discriminate texts
    - mainly functional role
    - In sentiment analysis, words like "not", "never", "no" must be kept since they flip meaning.
- Step 2: Remove low-frequency words
  - Typos
  - Very local information, **over-fitting**
  - Context matters, for instance in medical or legal texts

FOSTER SCHOOL OF BUSINESS

21

21

## Practice 4: Dimension Reduction

- Stop words

```
##Remove stop words
nltk.download('stopwords') #Downloads Stopword List
from nltk.corpus import stopwords
stop_words_removed = [token for token in token_d1 if token.isalpha() and token not in stopwords.words('english')]

print(token_d1)
print(stop_words_removed)
```

- Low-freq words

```
##low frequency words
from sklearn.feature_extraction.text import CountVectorizer
vectorizer2 = CountVectorizer(min_df=2)
vectorizer2.fit(c1)
print(vectorizer2.vocabulary_)
v2 = vectorizer2.transform(c1)
print(v2.toarray())
```

FOSTER SCHOOL OF BUSINESS

22

22

## Problem: Frequency

- Is frequency **a good representation** to discriminate text documents?
- That is, a document that is very different from the others will be separated more from the other documents.
- The Euclidean distance between that document and other documents will be longer.
- Question 1: Is a word more discriminating if it appears less or more times in **a document**? Does frequency capture this?

FOSTER SCHOOL OF BUSINESS

23

23

## Problem: Frequency

- Question 2: Is a word more discriminating if it appears in less or more documents? Does frequency capture this?
- Recall stop-words
- Consider a term in a document that is rare in the collection
  - e.g., *microeconometrics*
  - This term captures the uniqueness of the document, so it separates the document from those who don't contain it more.
  - We want a **higher weight** for rare terms, so the distance between this document and the rest will be longer.

FOSTER SCHOOL OF BUSINESS

24

24

## Example

- "capricious person"
  - occurrences of capricious count for much more in the final document ranking than occurrences of person.
- "try insurance"

| Word | Collection frequency | Document frequency |
|---|---|---|
| *insurance* | 10440 | 3997 |
| *try* | 10422 | 8760 |

  - Which word should get a higher weight?

FOSTER SCHOOL OF BUSINESS

25

25

## A Better Representation by Word Weighting

- The most popular weighting schema is **normalized** word frequency TF-IDF:
  - TF(t,d): (Normalized) Term Frequency (number of term t occurrences in one document d)
  - DF(t,D): (Normalized) Document Frequency (number of documents in collection D containing the term t)
  - TF-IDF: Term Frequency - Inversed Document Frequency

$$TF-IDF(t,d,D) = TF(t,d)\cdot \log \frac{N}{DF(t,D)}$$

- Relative importance of the word in the document
  - Increases with the number of occurrences within a document
  - Increases with the rarity of the term in the collection

FOSTER SCHOOL OF BUSINESS

26

26

## Practice 5: TFI-IDF

- Use a TF-IDF to replace frequency in the term-document matrix to represent the bag-of-words representation

```
##TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer3 = TfidfVectorizer()
vectorizer3.fit(c1)
print(vectorizer3.vocabulary_)

v3 = vectorizer3.transform(c1)
print(v3.toarray())
```

FOSTER SCHOOL OF BUSINESS

27

27

## Problem of Word-level: Lose Order Information

- He is **a good guy,** he is **not bad**
- He is **not a good guy,** he **is bad**

- Are they the same?

- **Local order information:** Phrase level information
  - Phrases as frequent contiguous word sequences
  - Phrases as frequent non-contiguous word sequences

FOSTER SCHOOL OF BUSINESS

28

28

## Text Representation: Phrase Level

29

29

## Preserve Phrase Level Information: n-grams

- To preserve phrase-level information, we can include token pairs, triplets, etc. in representations
  - 1-grams for words, tokens
  - 2-grams for two-word phrase, token pairs
  - n-grams for n-word phrases

- Bag of words -> Bag of n-grams

  - He is a **good guy**, he is **not bad**
  - He is not a **good guy**, he **is bad**

30

30

## Practice 6: Bag of 2-grams

- http://scikit-learn.org/stable/modules/feature_extraction.html

```
##Bag of 2-grams
vectorizer4 = TfidfVectorizer(ngram_range=(1, 2), min_df=2)

vectorizer4.fit(c1)
v4 = vectorizer4.transform(c1)

print(vectorizer4.vocabulary_)
print(v4.toarray())
```

FOSTER SCHOOL OF BUSINESS

31

31

## Problem 1: Too Many Dimensions

- Google n-gram corpus:
- http://googleresearch.blogspot.com/2006/08/all-our-n-gram-are-belong-to-you.html#links
  - File sizes: approx. 24 GB compressed (gzipped) text files
  - Number of tokens: 1,024,908,267,229
  - Number of sentences: 95,119,665,584
  - Number of unigrams: 13,588,391
  - Number of bi-grams: 314,843,401
  - Number of tri-grams: 977,069,902
  - Number of four-grams: 1,313,818,354
  - Number of five-grams: 1,176,470,663

FOSTER SCHOOL OF BUSINESS

32

32

## Problem 2: Still Lose Order Information

- I **drink** water in parties
- I grab a **drink** in parties

- **Part-of-speech level** order information matters!
  - Keep part-of-speech structure information before BoW **vectorization**

FOSTER SCHOOL OF BUSINESS

33

33

## Text Representation: Part-Of-Speech Level

FOSTER SCHOOL OF BUSINESS

34

34

## Preserve Part-of-speech Level Information

- Part-of-speech tag (POS)
- Tagging word-types enables us to differentiate word functions
- Classify words that somehow 'behave' alike:
  - Appear in similar contexts
  - Perform similar functions in sentences
  - Undergo similar transformations
- 9 conventional word classes of parts of speech

FOSTER SCHOOL OF BUSINESS

35

35

## Conventional Word Classes

| part of speech | function or "job" | example words | example sentences |
|---|---|---|---|
| Verb | action or state | (to) be, have, do, like, work, sing, can, must | EnglishClub **is** a website. I **like** EnglishClub. |
| Noun | thing or person | pen, dog, work, music, town, London, teacher, John | This is my **dog**. He lives in my **house**. We live in **London**. |
| Adjective | describes a noun | good, big, red, well, interesting | My dogs are **big**. I like **big** dogs. |
| Determiner | limits or "determines" a noun | a/an, the, 2, some, many | I have **two** dogs and **some** rabbits. |
| Adverb | describes a verb, adjective or adverb | quickly, silently, well, badly, very, really | My dog eats **quickly**. When he is **very** hungry, he eats **really** quickly. |
| Pronoun | replaces a noun | I, you, he, she, some | Tara is Indian. **She** is beautiful. |
| Preposition | links a noun to another word | to, at, after, on, but | We went **to** school **on** Monday. |
| Conjunction | joins clauses or sentences or words | and, but, when | I like dogs **and** I like cats. I like cats **and** dogs. I like dogs **but** I don't like cats. |

http://www.englishclub.com/grammar/parts-of-speech_1.htm

FOSTER SCHOOL OF BUSINESS

36

36

## POS Tag Examples

| verb |
|---|
| Stop! |

| noun | verb |
|---|---|
| John | works. |

| noun | verb | verb |
|---|---|---|
| John | is | working. |

| pronoun | verb | noun |
|---|---|---|
| She | loves | animals. |

| noun | verb | adjective | noun |
|---|---|---|---|
| Animals | like | kind | people. |

| noun | verb | noun | adverb |
|---|---|---|---|
| Tara | speaks | English | well. |

| noun | verb | adjective | noun |
|---|---|---|---|
| Tara | speaks | good | English. |

| pronoun | verb | preposition | adjective | noun | adverb |
|---|---|---|---|---|---|
| She | ran | to | the | station | quickly. |

| pron. | verb | adj. | noun | conjunction | pron. | verb | pron. |
|---|---|---|---|---|---|---|---|
| She | likes | big | snakes | but | I | hate | them. |

Here is a sentence that contains every part of speech:

| interjection | pron. | conj. | adj. | noun | verb | prep. | noun | adverb |
|---|---|---|---|---|---|---|---|---|
| Well, | she | and | young | John | walk | to | school | slowly. |

http://www.englishclub.com/grammar/parts-of-speech_1.htm

FOSTER SCHOOL OF BUSINESS

37

37

## A Finer Text Representation

- POS tagging maintains the order of words and their tags.
- However, if we use a Bag-of-Words representation, even with POS tags, we lose the order information.
- For some tasks, order matters more!
  - Translations
  - Voice recognition

FOSTER SCHOOL OF BUSINESS

38

38

## Penn Treebank Tag Set

| Tag | Description | Example | Tag | Description | Example |
|---|---|---|---|---|---|
| CC | Coordin. Conjunction | *and, but, or* | SYM | Symbol | +,%, & |
| CD | Cardinal number | *one, two, three* | TO | "to" | *to* |
| DT | Determiner | *a, the* | UH | Interjection | *ah, oops* |
| EX | Existential 'there' | *there* | VB | Verb, base form | *eat* |
| FW | Foreign word | *mea culpa* | VBD | Verb, past tense | *ate* |
| IN | Preposition/sub-conj | *of, in, by* | VBG | Verb, gerund | *eating* |
| JJ | Adjective | *yellow* | VBN | Verb, past participle | *eaten* |
| JJR | Adj., comparative | *bigger* | VBP | Verb, non-3sg pres | *eat* |
| JJS | Adj., superlative | *wildest* | VBZ | Verb, 3sg pres | *eats* |
| LS | List item marker | *1, 2, One* | WDT | Wh-determiner | *which, that* |
| MD | Modal | *can, should* | WP | Wh-pronoun | *what, who* |
| NN | Noun, sing. or mass | *llama* | WP$ | Possessive wh- | *whose* |
| NNS | Noun, plural | *llamas* | WRB | Wh-adverb | *how, where* |
| NNP | Proper noun, singular | *IBM* | $ | Dollar sign | *$* |
| NNPS | Proper noun, plural | *Carolinas* | # | Pound sign | *#* |
| PDT | Predeterminer | *all, both* | " | Left quote | (' or ") |
| POS | Possessive ending | *'s* | " | Right quote | (' or ") |
| PRP | Personal pronoun | *I, you, he* | ( | Left parenthesis | ( [, (, {, <) |
| PRP$ | Possessive pronoun | *your, one's* | ) | Right parenthesis | ( ], ), }, >) |
| RB | Adverb | *quickly, never* | , | Comma | , |
| RBR | Adverb, comparative | *faster* | . | Sentence-final punc | (. ! ?) |
| RBS | Adverb, superlative | *fastest* | : | Mid-sentence punc | (: ; ... – -) |
| RP | Particle | *up, off* | | | |

FOSTER SCHOOL OF BUSINESS

39

39

## Examples

E.g., IBM Corp. and Alvarion Inc. have established an alliance to offer wireless systems to municipalities and their public safety agencies, Alvarion announced.

IBM/NNP Corp./NNP and/CC Alvarion/NNP Inc./NNP have/VBP established/VBN an/DT alliance/NN to/TO offer/VB wireless/JJ systems/NNS to/TO municipalities/NNS and/CC their/PRP$ public/JJ safety/NN agencies/NNS ,/, Alvarion/NNP announced/VBD ./.

I/PRP **drink/VBP** water/NN in/IN parties/NNS

I/PRP grab/VBP a/DT **drink/NN** in/IN parties/NNS

**PRP: Personal Pronoun**
**VBP: Verb, non-3rd person singular present**
**NN: Noun, single**
**IN: Preposition**
**NNS: Noun, plural**

- Question1: would you do word normalization before or after POS tagging?
- Question2: would POS tagging increase dimensions of the BOW?

FOSTER SCHOOL OF BUSINESS

40

40

## Practice 7: POS Tagging

```
## POS TAG
d4 = "I drink water in parties"
d5 = "I grab a drink in parties"
token4 = nltk.word_tokenize(d4)

POS_token4 = nltk.pos_tag(token4)

c2 = [d4, d5]
POS_c2 = []
for doc in c2:
  token_doc = nltk.word_tokenize(doc)
  POS_token_doc = nltk.pos_tag(token_doc)
  POS_token_temp = []
  for i in POS_token_doc:
    POS_token_temp.append(i[0] + "_" + i[1])
  POS_c2.append(" ".join(POS_token_temp))
```

```
vectorizer5 = TfidfVectorizer()
vectorizer5.fit(POS_c2)
print(vectorizer5.vocabulary_)
```

```
POS_v5 = vectorizer5.transform(POS_c2)
print(POS_v5.toarray())
```

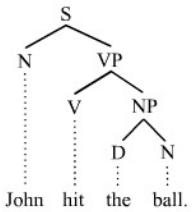
FOSTER SCHOOL OF BUSINESS

41

41

## An Ultimate POS-level Representation: Parse Tree

- Parsing provides maximum structural information per sentence
  - Fully keeps the order
- On the input we get a sentence, on the output we generate a parse tree




- Note:
  - Tree structure (larger dimension)
  - Rather than term-document matrix
  - Need recursive algorithm
  - Too complex for text analysis methods like topic modeling and classification

FOSTER SCHOOL OF BUSINESS

42

42

## Summary

- Text representation
  - Word level information
    - Tokenization
    - Bag of words
    - Stemming
    - Lemmatization
    - TF-IDF
  - Phrase level information
    - N-grams
  - Part-of-speech level information
    - POS
    - Parse Tree

FOSTER SCHOOL OF BUSINESS

43

43